TECHNISCHE HOCHSCHULE MITTELHESSEN

CAMPUS GIESSEN

MNI
Mathematik, Naturwissenschaften und Informatik

Master Thesis

Integration of model-driven development approaches into the domain of mobile applications

Classification of an interpreter implementation for mobile applications

Analysis of a model interpreter on the basis of an existing implementation

Reengineering of an model interpreter implementation

zur Erlangung des akademischen Grades

Master of Science

eingereicht im Fachbereich Mathematik, Naturwissenschaften und Informatik an der Technische Hochschule Mittelhessen

von

Samuel Schepp

March 20, 2019

Referent: Prof. Dr. Peter Kneisel

Korreferent: Dennis Priefer

Erklärung der Selbstständigkeit

Hiermit versichere ich, die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie die Zitate deutlich kenntlich gemacht zu haben.

Gießen, den March 20, 2019 Samuel Schepp

This thesis describes configuration-based mobile applications and their infrastructure. In addition, a possible way of implementing a configuration system is discussed. Finally, the implementation is compared to the approach of established literature by classifying the model and interpreter.

Furthermore, there is an evaluation of the use case of interpreting and generated apps.

Contents

1 Introduction 1
- 1.1 Problem definition . . . 1
- 1.2 Motivation . . . 2
- 1.3 Limitation . . . 3
- 1.4 Structure . . . 3
  - 1.4.1 Logical Approach . . . 4
  - 1.4.2 Goals . . . 4

2 Background 5
- 2.1 Company . . . 5
  - 2.1.1 Markets . . . 5
  - 2.1.2 City Marketing . . . 6
- 2.2 Model Interpreter System . . . 6
  - 2.2.1 Use Cases . . . 7
  - 2.2.2 Interoperability . . . 7
- 2.3 Models . . . 8
  - 2.3.1 General Model Theory . . . 8
  - 2.3.2 Specialized Model Theory . . . 9
- 2.4 Model Driven Development . . . 9
  - 2.4.1 Modeling a Software System . . . 10
  - 2.4.2 Code Generators . . . 12
  - 2.4.3 Model Interpreters . . . 12

3 Concept 13
- 3.1 Implementation Goals . . . 13
  - 3.1.1 Requirements . . . 13
  - 3.1.2 Drawbacks . . . 13
  - 3.1.3 Iterative Development . . . 13

4 Implementation 15
- 4.1 Language . . . 15
- 4.2 Components . . . 15
  - 4.2.1 Sub-Models . . . 15
  - 4.2.2 Layout . . . 15

4.2.3 Persistence 15
4.2.4 Interaction 15
4.3 Infrastructure 15
4.3.1 Control Center 15
4.3.2 Data Backend 15
4.3.3 Resource Server 15
4.3.4 Image Cruncher 15

5 Evaluation 17
5.1 Requirements 17
5.2 Analysis 17
5.2.1 Classification 17
5.2.2 Approach 17
5.2.3 Model 17

6 Conclusion 19
6.1 Summary 19
6.2 Discussion 19
6.3 Next Steps 19
6.3.1 Hybrid Approaches 19
6.3.2 Runtime Models 19
6.3.3 Abstraction 19

Bibliography 21

Index of abbreviations 23

List of Figures 25

List of Tables 27

Listings 29

1 Introduction

Mobile application and digital media usage is still on the rising[Per]. Mobile applications are one of many possible ways to interact with customers and consumers. A web appearance on social media or a dedicated web page can also fulfill these needs. The way, the user interacts with a web page is more shallow, because of lower expectations. A user needs to log in most of the time.

A mobile application can enable user specific features without logging in, because the user can be identified with an identification token, that is generated, while the user installs the app. It is possible to activate special features like geofencing, push notifications or local data storage, that is synchronized with the platform cloud services (e.g. Apple iCloud or Google Drive).

A mobile app is integrated more deeply into the device, than a web app. However, web browser already support a lot of the features by now, that originally were only available to native apps. The current development indicates, that the differences will be further eliminated by the browser vendors.

1.1 Problem definition

Mobile applications have significant disadvantages, that push the development of web apps forward. Changes have to be approved by the phone vendors, in order to be able to release updates to the app. This causes development delays and reduces the ability to react to events or customer requirement changes.

All apps have to be kept in good condition, which creates more work, the more apps there need to be maintained. New operating system features have to be implemented in all of the apps, which can cause inconsistencies. All apps have to be tested in detail, to make sure that all apps implement the required change correctly. Platform changes can only be dealt with one by one. Developers need deep system knowledge to develop the apps. Every change has to go through one of each platform experts, so development is expensive.

An example for a change, that all apps should implement is the display notch. It causes the screen to extend to the edge of the display. However some of the screen is cut out to make room for sensors and the camera.

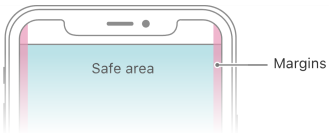

Figure 1.1: Display notch with safe area, Apple iPhone X

The notch is integrated to the default interface framework, so that developers can react accordingly and use the provided margins. This change requires all apps to implement a correct way to handle that. Developers have to go though every screen and correct the view.

1.2 Motivation

To be able to deliver the full mobile application experience, the app has to be native. This will change, when all of the phone's hardware interface is exposed to the web browser and user interface performance is better. But for now, the only way to use these features and be good on battery life and data usage is by providing a native app, that is written in the native language and uses features, directly provided by the phone vendors.

Web app wrapper like "ionic" are able to use these features and provide their own interface to the web app. In this way, a web app can in fact use restricted components like push notifications and beacons.

With this approach, every new app still has to be developed and old ones have to be maintained individually. In addition, it involves the dependence on third party framework vendors. Building apps with ionic creates a massive list of dependencies: ionic, angular, npm and nodejs. Native apps have the ability to reduce reliance, by only using native libraries, which are very comprehensive by default.

To be able to create many apps with little effort, a model driven approach can be considered. A well abstracted model can have the ability to be easy to understand by users and the platforms can bring their respected implementation in, so any app looks and feels native. A service can deliver pre-rendered sub-models, so that complex client workload can be moved to the server.

Existing apps are easy to maintain, because there is only one code base per platform. One bug fix applies to all apps. Also, it is easy for anyone to change small things like a

label, a background color or the sorting of a list. The goal is, to be able to configure new apps in one day, together with the customer.

1.3 Limitation

This thesis will not describe the development experience nor the initial expense of the project. Neither will this thesis evaluate financial aspects and at which point, the project was or is worth the development costs. The domain examination will be narrowed down to the two main ones, mobile phones and digital signage.

The thesis aims to answer the following questions:

> Which requirements are placed on the model interpreter implementation.
>
> How can the specific model interpreter be implemented.
>
> How can the implementation be evaluated on the basis of the requirements?

1.4 Structure

The thesis will give some information on the background of the interpreter. The company Fabrik19 GmbH, which is actively developing and maintaining an implementation, will be described and characterized. After that, the general concept of model driven development, code generators and interpreters are examined and presented.

The main content will then be about the concrete implementation, by describing the most significant design choices, overall infrastructure and usage examples. The focus will be on the two main platforms 'iOS' and 'Android'. The web and '.Net' clients are not considered, because they are not targeted for mobile devices and only support a subset of the model.

In the final chapter, the concept is applied to the implementation, to see how it fits the theoretical model of interpreters. Differences are worked out and discussed. Also, an outlook will be given, where additional topics, like hybrid approaches, runtime models and abstraction are covered.

1.4.1 Logical Approach

To be able to evaluate the interpreter, requirements are listed and described. Thees will be used to validate the implementation.

Since the implementation for the iOS platform has been re-engineered to meet current demands, the problems with the old version will be outlined and compared to the new one.

1.4.2 Goals

This thesis will show how a model interpreter can be evaluated against requirements and informal criteria. In addition, the re-engineering and model development processes will be compared to common approaches in modern literature.

2 Background

The system, that is discussed here, is a model interpreter that is designed to run mobile applications on iOS and Android. The system, which includes the interpreters and the backends are called mos[1]. It is in active development and powers over 50 apps that all are available in the Apple AppStore and Google PlayStore. The app categories range from retail to event to city marketing apps[2].

2.1 Company

The creator of the system is Fabrik 19 GmbH. The company is based in Gießen, Hesse, Germany and was founded 2013 by Mark Pralle and Sven Herchenhein. At the moment, the company has bout 25 employees. Besides of apps, there are also other projects and solutions, like consulting and web hosting.

2.1.1 Markets

Fabrik19 does not deliver apps directly to customers, but is partner of retail corporations and city marketing agencies. The current flagship is the Gießen App, which was developed in cooperation with DISTAMA GmbH. The app was downloaded 34,000 times and has 20 distinct and active users per day.

Besides mobile apps, digital signage is another important market. The companion to the Gießen App is a stele that contains a large touchscreen and runs an app that is optimized for big touchscreens. It is located on the main pedestrian shopping street in Gießen and provides respective informations on the stores and points of interests[3]. Other stele systems are also active in Globus hardware stores and Globus grocery stores.

1 Mobility Suite
2 `https://fabrik19.de/technologie/`
3 `https://distama.de/bid-seltersweg/`

2.1.2 City Marketing

Since the apps use live data and have the ability to change it's content structure, an active support and maintenance is needed to keep the app alive.

To be able to advertise and publish city marketing apps as a service, Distama GmbH was founded which is a subsidiary firm of Fabrik19. The goal was to combine technology knowledge with marketing, to be able to provide the app and the attached customer support. City marketing agencies are not always able to support an app, because they lack experience and domain knowledge. Distama is the mediator between the developer, the customer and the user.

2.2 Model Interpreter System

The main development focus lays on the app interpreter platform "mos". It supports the two major platforms iOS and Android, but there is also a lightweight version available in .Net, which is running on the steles as a Windows app. A web client is also actively in development, which aims to replace the .Net version.

A configuration are multiple JSON files, that represent the app meta information, its layout and its behavior. The files are called node. There is one root node, whose url is baked into the app while compiling. This root node defines the first screen of the app. All the other screens can be defined in other nodes. When displaying another node, the new node is loaded from cache or network and its new view is pushed on the view stack, which is done by the interpreter. On the particular platforms, this action will look different and will use the platforms native animations. Web apps like Ionic have their own push animations which makes them feel foreign. The previous node can pass data, like the ID of the item that should be displayed.

The goal is, to be able to create a new app in one day, by meeting with the customer and configuring the app together. The syntax of the configuration is designed to abstract all of an apps standard behavior. Because of a custom layout system, that can be described with constraints and will layout automatically, it is not possible to recognize that it is only a configured app, instead of a custom native one.

The apps have sophisticated caching and optimization mechanisms, which makes it possible to use them completely offline, without any additional configuration effort. The configuration, images and data are cached heavily, which makes the apps very fast. But because of the dynamic design of the configuration syntax, it is possible to clear the caches and force new versions. The cache first policy of the interpreter makes sure, that contents are always available and loads changes in the background.

Besides of displaying data, the client is highly intractable and can perform various custom tasks on the device, like uploading local data, saving data to the devices user settings and providing push, beacon and location infrastructure.

This data can be used in describing the layout by defining layer conditions, which makes the app behave accordingly.

2.2.1 Use Cases

The main use cases are marketing and customer interaction. The concept is data source driven and enables configuration of any data source. Because of a template engine, data can be arranged for display in an intuitive way.

Since the configuration can be changed on the fly and without the need of providing an update via app store, the contents can be adjusted to fit current events like holidays or promotions. Especially for digital signage it is important to be able to change the contents without any application updates.

Any change to the configuration is reflected as soon as the app checks for configuration changes. The update mechanism are also highly customizable, so that the optimal update intervals and strategies can be used for different apps.

A city marketing app might need a medium update interval, so that promotions and special content are available when needed. A event app should have a very short update interval, so that the events program changes or announcements are delivered as fast as possible.

2.2.2 Interoperability

The app can be filled with data via common REST services, that deliver JSON data. The app uses HTTP mechanisms and a custom delta system to only load new data. The mos backend system also has a data proxy, which can load any data and provide it in a simpler, preprocessed format, so it is simple for the apps to download. This way, it is possible to use any data source and still be able to change the templates and layouts without updates.

The apps have the capability to be integrated in a Wordpress instance, a chat service like Mattermost or an secured API that uses authentication methods like client certificates, authorization headers or JSON web tokens.

By default any network communication is secured with an app specific client certificate. Other authentication methods have to be configured.

2.3 Models

Since the mos-Framework tries to abstract mobile application platforms into a common language, the configuration could be considered to be an abstract model of a mobile application. To classify and evaluate the model approach for this domain, the following section will discuss models and their properties.

2.3.1 General Model Theory

A general model is described by Herbert Stachowiak[1]. He defines three properties, that apply to all models.

Abbildungsmerkmal (Representation): A model is a representation of an original, that is either a natural or artificial thing, that can have properties and behavior. The original can be a model it's self.

Verkürzungsmerkmal (Reduction): The second property is the ability to reduce properties and behavior of the original to a set, that is relevant for the model handlers. The original might be a complex system and only a subsystem or specific properties and their interaction are relevant. For this, the model can reduce the complexity, to be more simple to understand and to minimize the error rate.

Pragmatisches Merkmal (Pragmatism): Models can not be clearly assigned to their originals. That means, that original and model are not equal, but can be interchanged. There are three cases defined by Stachowiak, in which the model can be used as an replacement. Models are not only models of something, but they are also models for someone, a person or an artificial model user. They perform their functions within a time interval. And they are models for a specific purpose. Often times, the purpose is an action that is not possible on the original because it may be to complex, or an physical original could be destroyed.

If theses properties apply an object, i can be called model according to Stachowiak. It can be applied by numerous of common usages of the word. Mathematical models, philosophical models, simulations and concrete object models are some examples, that fit the requirements.

1 [Staa] 131

2.3.2 Specialized Model Theory

To classify models in a more specific way, they can be categorized as descriptive and prescriptive models. A descriptive model represents an existing system, as it describes it's current behavior. The general model specifications can still be applied, since all three properties are true for such a model. It cannot be modified unless the original changes, since it is representing it directly. A change to one of them without updating the other would result in the model not representing the original, which makes it a prescriptive model. A change on the original which is not relevant to the model will keep its validity.

Models, that represent a fictive object or system are called prescriptive models. The prefix "pre-" indicates, that the model exists before the original. The model is then used to prototype a system, and test their behavior and properties, without the need to create the full system.

According to Seidewitz 2003 [Sei] both descriptive and prescriptive models define a set of originals. That means, that a original can be derived from a model by adding additional information that does not contradict each other. Often times, the information that must be added is trivial or can be fetched from somewhere else. That is possible, because the model has by definition all of the necessary information.

A derived original fulfills all of the models properties, otherwise the original is not a correct derivation.

2.4 Model Driven Development

Models can be used in a software development process (MDSD[1]) or as part of a software architecture (MDA[2]). Software is also a model, which represents real life objects and procedures. Since it is well structured by default, it is fairly easy to create model from a existing code basis. UML[3] is a widely used modeling languages, which can represent static, dynamic and interactive aspects of software systems.

The problem is, that the domain specific structure of the code can be hard to understand, because the abstrcation level of a modern object oriented programming language is much lower, than the domain requires. VÖLTER and STAHL describe the result of scattering relevant information in the code as: 'The much-treasured inner structure is present in a

1 Model Driven Software Development
2 Model Driven Architecture
3 Unified Modeling Language

cloudy, distributed, and of course also a strongly individualized form.'[1] That means, that the domain's structure, components and behavior definition is represented in the software code, but in a way that is hard to understand. That is, because the relevant parts and the domain critical definitions are spread in the software's code. It is also harder to unterstand a domains properties, by only reading the programming language.

A software model, like an UML class diagram are an old tool to document and model software systems. The problem here is, that the abstraction level of an UML class diagram is often times the same as the programming language itself. A developer could find himself transforming a class into a diagram or generating a diagram, which does not contribute to the understanding of the domain structure. This happens, because the programming language itself is abstracted, but not the domain itself.

To address the problem of widely spread domain implementations, the programs important and relevant structure is compressed in a model, which defines and represents the program on a abstract and formal level. The model can be a textual DSL[2], a graphical diagram or even an informal text, that is then converted into a concrete syntax. It contains all the domain relevant information, which is required to derive a full implementation from it.

This procedure results in a model becoming part of the source of a program, and not only the documentation. A good model syntax can lead to the model being the documentation itself.

2.4.1 Modeling a Software System

An existing implementation can be analyzed, to create a model from it. If no model infrastructure exists, this is the way to approach the transformation.

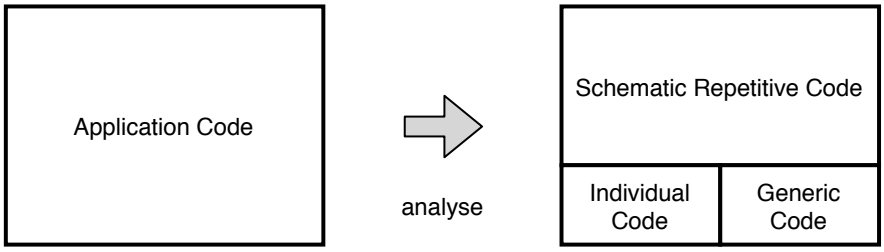

Figure 2.1: Analysing an existing implementation.

Figure 2.1 shows, how the code basis of an existing implementation of a software system can be split into three categories[3].

1 [Stab] 14
2 Domain Specific Language
3 [Stab] 15

The **schematic repetitive code** is the part, that needs to be abstracted into a model. It contains the application specific logic, that is later modeled by a user. It does not contain code, that is equal for all applications in the same domain.

Generic code is derived from the domain and target platform and not from the concrete application. Therefore, it is not modeled, but generated by the domain's MDD[1] infrastructure. It is mostly boilerplate code, that should not be part of the model. It is the same for every application and is later part of the generator templates.

Individual code is unable to be represented in the model, because it is to specific and does not fit into the domain. The tools need to make sure that this code can be integrated into the generated code later. To simplify the MDSD process, individual code should be avoided as far as possible, because in the worst case the individual code makes up all of the application code, which eliminates the possibility to use a model to represent the application. In the best case, the individual code portion is empty. This allows for a more simpler tools, because the application can be generated completely out of the model and there is no need for individual code interfaces.

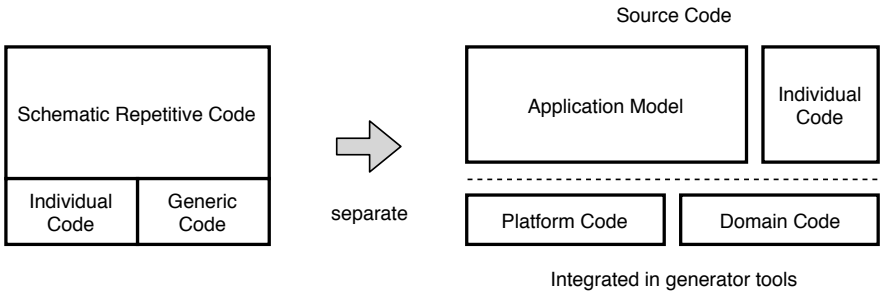

Figure 2.2: Separation of code into source code and steady code.

Figure 2.2 shows, how the analyzed parts can now be separated into source code and steady code. Source code describes the part of the code that is application specific and is developed by the MDD user. The application model and individual code should be checked into source control, since that is the actual source of the application, given a MDD infrastructure exists.

Platform and domain code is part of the MDSD tools. It is baked in, so that it is part of the output while generating. In case of code generators, this code would be part of the templates. In theory, it is now possible to develop any app, that fits into the domain, by only defining the model and the individual code. Extensions of the model's capabilities might need to be reflected in the platform or domain code.

1 Model Driven Development

2.4.2 Code Generators

2.4.3 Model Interpreters

3 Concept

3.1 Implementation Goals

Same day delivery

easy configuration

simple datasources

common app tasks simplified

content driven (wordpress, social media)

reactive form elements

3.1.1 Requirements

3.1.2 Drawbacks

boundary conditions

limits to development of app contents (app is out, but the customer needs more functionality, which is not possible: incomprehensible reasons for limits.)

3.1.3 Iterative Development

4 Implementation

lorem ipsum

4.1 Language

4.2 Components

4.2.1 Sub-Models

4.2.2 Layout

4.2.3 Persistence

4.2.4 Interaction

4.3 Infrastructure

4.3.1 Control Center

4.3.2 Data Backend

4.3.3 Resource Server

4.3.4 Image Cruncher

5 Evaluation

lorem ispum

5.1 Requirements

5.2 Analysis

lorem ispum

5.2.1 Classification

5.2.2 Approach

5.2.3 Model

6 Conclusion

6.1 Summary

6.2 Discussion

6.3 Next Steps

6.3.1 Hybrid Approaches

6.3.2 Runtime Models

6.3.3 Abstraction

Bibliography

[Per] Perkins, Kleiner: Internet Trends Report 2018, URL https://www.kleinerperkins.com/perspectives/internet-trends-report-2018/

[Sei] Seidewitz, Ed: What Models Mean. URL https://www.researchgate.net/publication/3248044_What_models_mean

[Staa] Stachowiak, Herbert: *Allgemeine Modelltheorie*, URL http://archive.org/details/Stachowiak1973AllgemeineModelltheorie

[Stab] Stahl, Thomas und Völter, Markus: *Model-driven software development: technology, engineering, management*, John Wiley, OCLC: ocm64453466

Index of abbreviations

mos Mobility Suite

DSL Domain Specific Language

MDA Model Driven Architecture

MDD Model Driven Development

MDSD Model Driven Software Development

UML Unified Modeling Language

List of Figures

1.1 Display notch with safe area, Apple iPhone X 2

2.1 Analysing an existing implementation. 10
2.2 Separation of code into source code and steady code. 11

List of Tables

Listings

.